pynxtools-xps – parser details

← Back to overview

PARSER & LINKS						IMPLEMENTED BEHAVIOUR			REVIEW
Parser	**Input format**	**Vendor site**	**GitHub**	**Docs**	**Metadata map**	**Standard behaviour**	**Special behaviour**	**Formula / transformation**	**Notes – issues to check**
Scienta Omicron	Plain text (.txt), Igor Binary Wave Format (.ibw), and HDF5 (.h5 / .hdf5)	wavemetrics.com	GitHub	Docs	metadata.py	Three readers (HDF5, Igor .ibw, TXT) share metadata.py/data_model.py. HDF5: energy axis from acquisition/spectrum/data/x_axis (index 0..N−1 if missing), extra *_axis datasets become additional dims. Igor: axis = sfA·i + sfB per wave dimension; a dimension named kinetic_energy, binding_energy or analyser_energy is renamed "energy" and sets energy_scale; intensity is the first data label without "reduced". TXT: energy and intensity are the two columns of the data section. Output dims (cycle, scan, energy) with cycle = scan = 1 (HDF5 may add axes). Header fields mapped (low_energy→start_energy, high_energy→stop_energy, energy_step→step_size, dimension_size→energy_size, dimension_scale→energy_axis) with eV defaults.	The .txt extension is shared with the CasaXPS text-export parsers (vms_export/parser_export.py:110, 581). The reader tells them apart by the first line of the file: Scienta requires exactly [Info] (scienta/txt_parser.py:55), CasaXPS rows layout requires Characteristic Energy eV and Acquisition Time s, and CasaXPS columns layout requires a line starting with Cycle and containing : (vms_export/parser_export.py, matches_file of each layout). A given file can match only one.	HDF5: E = x_axis, or E_i = i (i = 0..N−1) if missing (hdf5_parser.py l. 159-164). Igor: E_i = sfA·i + sfB, i = 0..nDim−1 (igor_parser.py l. 268-271). TXT: E = first column of the data lines (txt_parser.py l. 192-193, 209). Header scale: np.array([float(s) for s in scale.split(" ")]) (metadata.py l. 81). No hv − KE, no flip, no work function. Intensity: HDF5 data[np.newaxis, np.newaxis, ...]; Igor first non-"reduced" label, squeezed, (np.newaxis, np.newaxis, :]; TXT second column, same shape.	(1) Igor uses the first axis label as "energy" if no energy dimension exists, then an empty array (igor_parser.py l. 181-185). (2) Igor .squeeze() + [np.newaxis, np.newaxis, :] only fits 1D data (l. 193, 205). (3) HDF5 index-axis fallback is in channels but may be labelled eV (hdf5_parser.py l. 161-163; metadata.py l. 218). (4) HDF5 gets no energy coordinate if n_physical > len(data_axes) (l. 174-175, 181). (5) Default unit lookup uses "energy" but _DEFAULT_UNITS has "energy_axis" (igor_parser.py l. 150; metadata.py l. 218). (6) TXT split(" ") and float() drop tab-separated lines, decimal commas and extra columns silently (txt_parser.py l. 192, 195-196). (7) TXT header scale and data axis are never compared; _separate_dimension_scale fails on double or trailing spaces (metadata.py l. 81). (8) Header range (start_energy, stop_energy, step_size) not checked against the axis. (9) energy_scale, energy_scale_2 and energy_mode can disagree; data-model default "kinetic" is stored in a field called energy_units (data_model.py l. 47). (10) "analyser_energy" is neither binding nor kinetic (igor_parser.py l. 144). (11) No work function or BE/KE conversion in this folder. (12) Leftover print(entry_name) (igor_parser.py l. 175).
SPECS	SpecsLabProdigy and SpecsLab 2 files from SPECS GmbH (.sle, .xml, .xy)	—	GitHub	Docs	metadata.py (SLE)	Maps SPECS metadata to NeXus names (Ebin, Ekin, Epass, Eexc, wf) with eV units. Energy axis from start, step and points (BE: start − i·step; KE: start + i·step). Data reshaped to (n_energy, n_channels), stored as (cycle, scan, energy) or (cycle, scan, channel, energy). Transmission function KE axis: KE = excitation_energy − BE; flat TF of ones if absent.	FAT mode only: each channel shifted by detector_shift E_pass, then linearly interpolated onto the common overlap grid (output step can differ from step_size). Channels summed (np.sum). Raw axis is relative (i·step_size). BE axis flipped with np.flip. ValueError if step_size or energy axis info is missing.	Raw axis: E_i = i·step_size (i=0..N-1). FAT mode: E_i,c = i·step_size + shift_c (shift_c = detector_shift_c·E_pass); common grid E_j = xmin + j·(xmax−xmin)/(num_values−1), xmin = max_c(min E_i,c), xmax = min_c(max E_i,c); linear interp1d per channel; merged_i = Σ_c I_i,c. BE axis: np.flip(E_calib). Axis from metadata: BE_i = BE_start − i·step; KE_i = KE_start + i·step. TF axis: KE = excitation_energy − BE (BE type), else KE = E.	(1) np.flip applied to energy axis only, not to interpolated data, so BE spectra may be mirrored. (2) Raw axis is relative (starts at 0); check where BE_start/KE_start is added and its sign. (3) Output step differs from step_size when shifts are applied or num_values ≠ N. (4) Shift sign identical for BE and KE. (5) Assumes detector shifts are fractions of E_pass. (6) Shift correction only in fixed_analyzer_transmission mode. (7) work_function parsed but not used in KE = hv − BE.
					metadata.py (XML)	Reads SPECS XML; per region collects excitation_energy, kinetic_energy, kinetic_energy_base, pass_energy, scan_delta, MCD shifts/head/tail and scan mode. The binding-energy axis is computed, not read. Counts are de-interleaved per channel (scan_counts[row::mcd_num]) and stored as (cycle, scan, energy) summed, plus (cycle, scan, channel, energy) raw.	FAT mode: channels are only trimmed (mcd_head/mcd_tail) and summed; no per-channel shift or interpolation. Non-FAT modes: each channel starts at index offset_id = round(offset/scan_delta) (minus 1 if > 0), first N points taken, then summed. Common axis is the last channel's axis (offset 0). Axis rounded to 8 decimals. Most metadata/unit mappings are commented out in metadata.py.	BE_upper = hv − KE + KE_base + 1; offset_c = (shift_last − shift_c)·E_pass; start_c = BE_upper − offset_c; end_c = start_c − N·scan_delta; axis_c = linspace(start_c, end_c, N) (N = values_per_scan); BE = axis_last = linspace(BE_upper, BE_upper − N·scan_delta, N) (descending); counts_sum = Σ_c counts_c	(1) Hard-coded +1 in BE_upper = hv − KE + KE_base + 1 (l. 665-667): check with a reference peak (e.g. Au 4f7/2). (2) Work function not applied; mapping commented out (metadata.py l. 40). (3) linspace span N·scan_delta with endpoint included gives step N·Δ/(N−1) ≠ scan_delta (l. 683-691). (4) FAT mode sums channels with no MCD shift correction, only head/tail trim (l. 720-728); SLE interpolates. (5) offset_id − 1 asymmetry and rounding to whole points in non-FAT mode (l. 676-677). (6) BE axis assumed always; no energy/@type handling. (7) Unit/metadata mappings commented out (metadata.py l. 27-53).
					metadata.py (XY)	Reads the energy axis directly from the first data column of the file (no computation). Scans grouped by loop_no (cycle) and sorted by scan_no; intensities (2nd column) stacked to (n_cycles, n_scans, n_energy) with dims (cycle, scan, x_axis_name). Header fields mapped to NeXus names (ex_energy→excitation_energy, binding_energy→start_energy, eff._workfunction→work_function, scan_mode, energy_axis→x_units for binding/kinetic type), with float conversion and eV units. Step size measured from the axis if spacing is uniform.	No BE/KE conversion: the axis stays in the file's scale, with no flip, and excitation_energy and work_function are parsed but not used in any calculation. One axis (from the first scan) for all cycles and scans. Metadata taken from the first scan only. Other data columns stored as external_{name} arrays (n_cycles, n_scans, n_energy). Coordinate name is the first column's name, not necessarily "energy".	No energy formula: axis read from file, E = first data column of scan 0 (l. 266), used for all scans. Intensity: np.stack per cycle → (n_scans, n_energy), then all cycles → (n_cycles, n_scans, n_energy) (l. 268-275); no sum, mean, reshape or flip. step_size = _get_minimal_step(E) only if spacing is uniform (l. 790-793), metadata only.	(1) Dim/coord name uses the first column's name, but step_size looks up the literal key "energy" (l. 263 vs 790), so step_size may never be set; docstrings also say "energy" (l. 92, 124, 227). (2) Intensity is always the 2nd column, whatever its name (l. 264). (3) np.stack fails on ragged scans (l. 269-275). (4) Axis from scan 0 only; later axes not compared (l. 266). (5) zip silently drops extra values or labels on malformed data lines (l. 784). (6) Metadata from the first scan only (l. 286-296). (7) Work function unused; start_energy set from "binding_energy" regardless of axis type (metadata.py l. 83-84).
PHI Electronics	.spe (single spectra) and .pro (sputter profile / external parameter scan) files from PHI Electronics VersaProbe 4 instruments	phi.com	GitHub	Docs	metadata.py	Reads PHI region definition strings by position (spectrum_type, n_values, step, start, stop, dwell_time in s, pass_energy in eV) and builds the energy axis from start/stop/step (not read directly). Energy type is set to "binding" and units to eV. All scans of a region are stacked in file order into (1, n_scans, n_energy), dims (cycle, scan, energy). Header fields mapped to NeXus names (analyser_work_fcn→analyzer_work_function, energy_recal→energy_recalibration, analyser_mode→energy_scan_mode, neutralizer_energy→flood_gun_energy, energy_reference→peak + energy in eV).	Energy type "binding", energy units "eV" and dwell-time unit "s" are hard-coded (parser.py l. 315-318), not read from the file. Step is negated before use (l. 311) and the axis is reversed with np.flip (l. 320). n_values is read but not passed to the axis function. Scans are stacked as is, with no sum, mean or flip of intensities; cycle dimension is always length 1. Header lines are matched in order against the first remaining line only (l. 295-305). energy_reference and energy_recalibration are stored as metadata but not applied to the axis. "energy" and "data" are excluded from metadata (l. 55).	step = −float(def_split[5]); start = float(def_split[6]); stop = float(def_split[7]) (l. 311-313). Axis: E = np.flip(safe_arange_with_edges(stop, start, step)) (l. 320); no hv − KE, no work function. Data: scans stacked to (n_scans, n_energy), then np.newaxis → (1, n_scans, n_energy) (l. 131-134). pass_energy = float(def_split[-2]); dwell_time = float (def_split[-3]).	(1) "binding" hard-coded (l. 317); kinetic-energy files would be mislabelled. (2) Negated step, (stop, start, step) argument order and np.flip (l. 311, 320): check final direction and that first/last values equal start/stop; confirm intensities (file order) match the flipped axis with a known peak. (3) n_values unused (l. 310): check len(energy) == n_values. (4) Position-based parsing of region_definition (l. 309-318): layout change gives wrong values or errors. (5) Header lines matched in order (l. 295-305): a missing optional key shifts matching. (6) energy_reference / energy_recalibration not applied to the axis. (7) Work function parsed but unused (metadata.py l. 141). (8) Energy reference string split on a space (metadata.py l. 73); ValueError if not exactly two parts. (9) Ragged scans in np.array (l. 131).
VAMAS ISO	VAMAS (.vms, .npl) files	—	GitHub	Docs	metadata.py	Reads VAMAS blocks and builds the energy axis by scan mode: REGULAR from abscissa_start, abscissa_step and num; IRREGULAR from the interleaved abscissa values in the data; MAPPING handled separately. Data stacked in file order from the first scan's axis into (1, n_scans, n_energy) (y_cps) and (1, n_scans, 1, n_energy) (raw counts). Header fields mapped (source_energy→excitation_energy, resolution→pass_energy, abscissa_label→energy_label, abscissa_start→start_energy) with eV units.	Default axis label is "kinetic energy"; conversion to binding energy (x = source_energy − x; abscissa_start reset) is available but no call site is visible in this folder. Binding axes are reversed with np.flip when the label is exactly "binding energy". CasaXPS energy calibrations from the block's comments shift x after parsing (BE: x = offset; KE: x + offset), only for operation "ADD"; range calibration is a TODO. Axis and metadata come from the first scan (sorted by scan_no); cycle and channel dimensions have length 1.	Regular: E_i = round(start + i·step, 2), i = 0..num−1, num = num_ord_values/no_variables, flipped if label is "binding energy" (l. 431-437). Irregular: E = block_data[:, no_variables + 1], flipped if binding; abscissa_start = min(E), abscissa_step = minimal step (l. 467-473). y_var = data_array[var :: no_variables] (l. 457). KE→BE: x = source_energy − x (data_model.py l. 152). Casa: offset = min (measured_energies) − aligned_energy; BE: x − offset, KE: x + offset (data_model_casa.py l. 319, 501, 504).	(1) np.flip applied to x but not to y in regular mode (l. 436-437, 447-459); irregular y extraction not seen. (2) apply_energy_shift returns None for non-ADD operations or unknown energy type, so block.x becomes None (data_model_casa.py l. 498-507, parser.py l. 597); range calibration treated as simple shift by min(measured). (3) KE sign (x + offset) is right only if measured/aligned are binding energies. (4) Work function ignored in KE→BE (data_model.py l. 152). (5) No visible caller of convert_to_binding_energy_scale. (6) round(…, 2) breaks steps below 0.01 eV. (7) Exact label match "binding energy". (8) abscissa_start = min(E) differs from the file's start for flipped axes. (9) resolution mapped to pass_energy in eV even in retarding-ratio mode.
Kratos	VAMAS ISO 14976 format files (.vms)	—	GitHub	Docs	metadata.py	Metadata-only parser: reads Kratos header comment lines ("key : value") passed in from the VAMAS comment handler, normalises keys (start→energy_start, end→energy_end, centre→energy_centre, width→energy_width, tilt→sample_tilt, lens→lens_mode), converts values to the dataclass field types (dates, description and deflection tuples, booleans), sets eV units for energy fields, and flattens the result (_units → /@units). No spectrum data or energy axis is built.	_parse is a stub (pass) and matches_file always returns True (parser.py l. 52, 78); only used through vms/comment_handler.py. Lines without exactly one " : " and unknown keys are skipped silently. Energy axis, binding/kinetic type, excitation and pass energy come from the VAMAS parser, not from here.	None: no axis construction, conversion, flip or sum. Header line → key, raw_value = line.split(" : "); key normalised via _KEY_MAP; value = field_type(value), with field_type taken from the dataclass default (float for energy fields, int for number_steps); energy units "eV" from data-model defaults; _units → /@units on flatten (parser.py l. 95-110, 134-151).	(1) matches_file always True (l. 52) would claim every file if registered. (2) _parse is pass (l. 78). (3) Only the split is inside try (l. 94-98): ValueError from converters, dates or int("301.0") ends parsing of the whole comment block unless the VAMAS handler catches it. (4) Values containing " : " are dropped silently. (5) step_size and number_steps are not in _KEY_MAP; check they are filled on a real file. (6) Types follow the field defaults. (7) description values are strings, deflection values are floats (metadata.py l. 47-52 vs 69-72). (8) Regex has no exponent or trailing-dot support (l. 34-36, 62). (9) Energy units stored as given, not converted. (10) No cross-check of energy_start/end/step_size/number_steps against the VAMAS axis. (11) Defaults of 0.0 are indistinguishable from real zero values.